

Jaypee Institute of Information Technology, Noida

DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING AND
INFORMATION TECHNOLOGY



Project Title : Benchmarking Reinforcement Learning Algorithms
for Adaptive Traffic Signal Control

Enrol. No.	Name of Student
-------------------	------------------------

9923103105	Sachit Rai
------------	------------

9923103063	Garv Raj
------------	----------

9923103104	Ansh Tyagi
------------	------------

Course Name: Minor Project-2

Course Code: 15B19CI691

Program: B. Tech. CSE

3rd Year 6th Sem

2025 - 2026

Abstract

This project develops and benchmarks three reinforcement learning algorithms for adaptive traffic signal control at a single signalized intersection, using SUMO (Simulation of Urban MObility) as the simulation environment. The core problem addressed is the inefficiency of fixed-cycle traffic signals, which operate on predetermined timers regardless of real-time traffic conditions.

The project frames traffic signal control as a Markov Decision Process (MDP) in which an agent observes intersection state and selects signal phases at regular intervals, receiving a reward based on the change in mean vehicle waiting time. Three algorithms are trained and compared: Q-Learning (tabular baseline), Deep Q-Network (DQN), and Proximal Policy Optimization (PPO). A fixed-cycle timer serves as the naive baseline against which all learned policies are evaluated.

A Python-based training orchestrator manages simultaneous multi-process training, with live metrics written to CSV files and rendered in a real-time dashboard. Final comparison charts are generated post-training. The project claims PPO will outperform DQN will outperform Q-Learning on mean vehicle waiting time, and that all three will outperform the fixed-cycle baseline — consistent with published benchmarks from the sumo-rl library. The contribution is a clean, reproducible implementation with proper instrumentation and a systems-level orchestration layer not present in the existing ecosystem.

Keywords: Reinforcement Learning, Traffic Signal Control, SUMO, Q-Learning, DQN, PPO, Adaptive Control, Markov Decision Process

Acknowledgement

We would like to express our sincere gratitude to our supervisor, Dr. Neeraj Pathak, Professor, Department of Computer Science and Engineering, Jaypee Institute of Information Technology, Noida, for valuable guidance, encouragement, and constructive feedback throughout this project.

We are also thankful to our faculty members and peers for their continuous support and cooperation. We extend our appreciation to the developers of SUMO and the sumo-rl library for providing an open-source, research-grade traffic simulation platform. Finally, we extend heartfelt gratitude to our families for their motivation and encouragement.

Signatures of Students

Sachit Rai (9923103105)

Garv Raj (9923103063)

Ansh Tyagi (9923103104)

Declaration

We hereby declare that this submission is our own work and that, to the best of our knowledge, it contains no material previously published or written by another person nor material accepted for the award of any degree from any other institution, except where due acknowledgment has been made.

Place: Noida

Date: 20/03/2026

Sachit Rai (9923103105)

Garv Raj (9923103063)

Ansh Tyagi (9923103104)

Certificate

This is to certify that the work titled **Benchmarking Reinforcement Learning Algorithms for Adaptive Traffic Signal Control** submitted by Sachit Rai, Garv Raj, and Ansh Tyagi of B.Tech in Computer Science and Engineering has been carried out under my supervision. This work has not been submitted to any other university for the award of a degree or diploma.

Signature of Supervisor

Dr. Neeraj Pathak
Professor

Date: 20/03/2026

Contents

Abstract	i
Acknowledgement	ii
Declaration	iii
Certificate	iv
Table of Contents	vi
List of Figures	vii
List of Abbreviations	viii
List of Nomenclature	ix
1 Introduction	1
2 Literature Review	2
2.1 Fixed-Cycle vs. Adaptive Signal Control	2
2.2 Reinforcement Learning for Traffic Signal Control	2
2.3 SUMO as a Research Platform	2
3 Background Study	4
3.1 The Traffic Signal Control Problem as an MDP	4
3.2 Q-Learning	4
3.3 Deep Q-Network (DQN)	4
3.4 Proximal Policy Optimization (PPO)	5
3.5 The SUMO Simulation Environment	5
4 Requirement Analysis	6
4.1 Functional Requirements	6
4.2 Non-Functional Requirements	6
4.3 Challenges	6
5 Methodology	8
5.1 Environment Configuration	8
5.2 Proposed Algorithm Configurations	8
5.2.1 Q-Learning	8
5.2.2 Deep Q-Network (DQN)	8
5.2.3 Proximal Policy Optimization (PPO)	9
5.2.4 Fixed-Cycle Baseline	9

5.3	Proposed Training Orchestration	9
5.4	Evaluation and Visualization	10
5.5	Proposed Workflow	11
6	Implementation Plan	12
6.1	Technology Stack	12
6.2	Proposed Q-Learning Module	12
6.3	Proposed DQN Module	12
6.4	Proposed PPO Module	13
6.5	Proposed Training Orchestrator	13
6.6	Proposed Post-Training Analysis	13
7	Expected Results and Analysis	14
7.1	Hypothesized Performance Ordering	14
7.2	Learning Curve Analysis	14
7.3	Evaluation Metrics	14
7.4	Comparison with Published Benchmarks	15
8	Conclusion and Future Scope	16
8.1	Conclusion	16
8.2	Future Scope	16

List of Figures

5.1	Proposed System Architecture and Training Pipeline	10
5.2	Proposed End-to-End Workflow of the Training Pipeline	11

List of Abbreviations

AI	Artificial Intelligence
CSV	Comma-Separated Values
DLR	German Aerospace Center (Deutsches Zentrum für Luft- und Raumfahrt)
DQN	Deep Q-Network
MDP	Markov Decision Process
ML	Machine Learning
MLP	Multi-Layer Perceptron
PPO	Proximal Policy Optimization
RL	Reinforcement Learning
ReLU	Rectified Linear Unit
SUMO	Simulation of Urban MObility
RLHF	Reinforcement Learning from Human Feedback
API	Application Programming Interface

List of Nomenclature

English Symbols

s	State observed by the agent
a	Action selected by the agent (signal phase)
r	Reward signal received after action
$Q(s, a)$	Expected cumulative reward for state-action pair
π	Policy: mapping from states to actions
γ	Discount factor for future rewards
α	Learning rate
ϵ	Exploration rate (epsilon-greedy policy)
θ	Neural network parameters
W_t	Mean vehicle waiting time at timestep t
ΔW	Change in mean waiting time (reward signal)

Chapter 1

Introduction

Traffic signal control is one of the most pervasive optimization problems in urban infrastructure. Signals in most cities operate on fixed-cycle timers: they rotate through phases at predetermined intervals regardless of whether any vehicles are present. The consequence is measurable and daily — a red light holds twenty cars waiting while the crossing road is empty, or a green phase expires before a queue of vehicles can clear. Summed across every intersection in a city over a year, the wasted time and fuel represent a significant economic and environmental cost.

The core question this project investigates is whether a learning agent can discover a superior signal control policy purely from experience, with no hardcoded rules about traffic flow. If the agent only sees the current state of the intersection and a reward signal reflecting whether waiting time increased or decreased, can it learn when to switch phases and when to hold them?

This project frames traffic signal control as a reinforcement learning (RL) problem and benchmarks three algorithms — Q-Learning, Deep Q-Network (DQN), and Proximal Policy Optimization (PPO) — against each other and against a fixed-cycle timer baseline. The simulation environment is SUMO (Simulation of Urban MObility), a real traffic simulator developed by the German Aerospace Center (DLR) and used in academic and industry research. The `sumo-rl` library wraps SUMO into a standard Gymnasium interface, allowing RL algorithms to interact with the simulation through a clean step-reward API.

The project’s scientific claim is that PPO will achieve the lowest mean vehicle waiting time, followed by DQN and then Q-Learning, with all three surpassing the fixed-cycle baseline. This ordering is consistent with published results from the `sumo-rl` benchmark paper. The novel contribution is a clean, reproducible implementation with proper instrumentation: a multi-process training orchestrator, live metrics dashboard, and post-training comparison charts that together treat ML training as an infrastructure problem.

Chapter 2

Literature Review

2.1 Fixed-Cycle vs. Adaptive Signal Control

Traditional traffic signal controllers operate on pre-timed plans derived from historical traffic surveys. Webster’s method (1958) provides the foundational formula for optimal cycle lengths under steady-state assumptions, but real traffic is non-stationary — demand fluctuates by time of day, day of week, and in response to incidents. Actuated controllers that use loop detectors to extend green phases represent an early form of adaptation but still rely on hand-tuned thresholds and cannot optimize globally across multiple phases.

SCATS and SCOOT are the most widely deployed adaptive systems and improve on fixed-cycle plans by adjusting phase splits and offsets in real time. However, both systems are model-based and require careful calibration. Reinforcement learning offers a model-free alternative that learns directly from interaction.

2.2 Reinforcement Learning for Traffic Signal Control

The application of RL to traffic signal control dates to the early 1990s. Wiering et al. (2004) demonstrated that multi-agent Q-Learning could reduce average travel times in a grid network compared to fixed plans. The central challenge is state space size: a tabular Q-table requires storing a value for every possible (state, action) pair, which becomes intractable as the number of lanes, vehicles, and phases grows.

Deep Q-Networks, introduced by Mnih et al. (2015) for Atari game playing, addressed this by approximating the Q-function with a neural network. The two key stabilization innovations — experience replay and a target network — made deep RL practical. Subsequent work applied DQN to traffic control, showing consistent improvements over both fixed-cycle and actuated baselines.

Wei et al. (2018) introduced IntelliLight, a DQN-based controller that used a rich state representation including a queue length image and achieved significant waiting time reductions on real traffic datasets from Hangzhou. Li et al. (2021) proposed PressLight and MPLight, extending the approach to multi-intersection coordination using a pressure-based reward signal.

Policy gradient methods, particularly PPO (Schulman et al., 2017), have more recently been applied to traffic control. Unlike value-based methods, policy gradient methods directly optimize the policy and handle the non-stationarity of traffic demand more naturally through on-policy updates. The sumo-rl benchmark paper reports that PPO achieves the best per-episode reward on the single-intersection task, followed by DQN and Q-Learning — the ordering this project seeks to reproduce and validate.

2.3 SUMO as a Research Platform

SUMO is an open-source microscopic traffic simulator capable of modelling vehicle physics, lane changes, traffic light control, and pedestrian movement. It is used by BMW, DLR, and numerous academic groups

for traffic research. The `sumo-rl` library, developed by Alegre et al., wraps SUMO intersections as Gymnasium environments, enabling direct integration with standard RL libraries such as `stable-baselines3` and `RLlib`. This project uses `sumo-rl` as the environment layer.

Chapter 3

Background Study

3.1 The Traffic Signal Control Problem as an MDP

A Markov Decision Process is defined by a tuple (S, A, T, R, γ) where S is the state space, A the action space, T the transition function, R the reward function, and $\gamma \in [0, 1)$ the discount factor.

In the traffic signal control formulation used by this project:

- **State s_t :** At each 5-second decision step, the agent observes the current signal phase and the queue length (number of waiting vehicles) on each approach lane.
- **Action a_t :** The agent selects which signal phase to activate from the set of valid phases for the intersection.
- **Reward r_t :** The negative change in mean vehicle waiting time: $r_t = -(W_t - W_{t-1})$. A reward of zero means waiting time held constant; a positive reward means it decreased.
- **Episode:** A single simulation run of fixed duration (typically 3600 simulation seconds).

3.2 Q-Learning

Q-Learning (Watkins and Dayan, 1992) is an off-policy temporal difference algorithm that maintains a table $Q(s, a)$ representing the expected cumulative discounted reward from taking action a in state s and following the optimal policy thereafter. The update rule is:

$$Q(s_t, a_t) \leftarrow Q(s_t, a_t) + \alpha \left[r_t + \gamma \max_{a'} Q(s_{t+1}, a') - Q(s_t, a_t) \right] \quad (3.1)$$

where α is the learning rate and γ is the discount factor. Exploration is managed by an ϵ -greedy policy that selects a random action with probability ϵ and the greedy action otherwise, with ϵ decaying over training.

The fundamental limitation of tabular Q-Learning for traffic control is the state space explosion. Even discretizing queue lengths into a small number of bins per lane produces a table too large to populate adequately with experience. This makes Q-Learning an existence proof that naive approaches fail, rather than a practical solution.

3.3 Deep Q-Network (DQN)

DQN replaces the Q-table with a neural network $Q(s, a; \theta)$ parameterized by weights θ . The network takes the state vector as input and outputs Q-values for all actions simultaneously. The loss minimized is:

$$\mathcal{L}(\theta) = \mathbb{E}_{(s, a, r, s') \sim \mathcal{D}} \left[\left(r + \gamma \max_{a'} Q(s', a'; \theta^-) - Q(s, a; \theta) \right)^2 \right] \quad (3.2)$$

where $\mathcal{D}$ is a replay buffer of past transitions and θ^- are the parameters of a periodically-updated target network. These two innovations address the instability of training a neural network on correlated, non-stationary targets.

3.4 Proximal Policy Optimization (PPO)

PPO is a policy gradient algorithm that directly optimizes the policy $\pi_\theta(a|s)$ — the probability distribution over actions given a state. Rather than learning which states are valuable (as DQN does), PPO learns which actions to take. The key innovation is a clipped surrogate objective that prevents excessively large policy updates:

$$\mathcal{L}^{\text{CLIP}}(\theta) = \mathbb{E}_t \left[\min \left(r_t(\theta) \hat{A}_t, \text{clip}(r_t(\theta), 1 - \epsilon, 1 + \epsilon) \hat{A}_t \right) \right] \quad (3.3)$$

where $r_t(\theta) = \pi_\theta(a_t|s_t)/\pi_{\theta_{\text{old}}}(a_t|s_t)$ is the probability ratio and $\hat{A}_t$ is the estimated advantage. PPO is on-policy: it trains only on experience collected under the current policy, then discards it. This makes it better suited to non-stationary environments like traffic, where the distribution of states changes as the policy improves.

PPO is notable for its use at scale: it is the algorithm behind OpenAI Five (Dota 2), OpenAI’s RLHF training for InstructGPT and ChatGPT, and many robotics control applications.

3.5 The SUMO Simulation Environment

SUMO models vehicle movement at the microscopic level: each vehicle has an individual car-following model (Krauss model by default) governing its acceleration and braking, and a lane-change model. Traffic lights are represented as state machines with configurable phase sequences. The `sumo-rl` library exposes a single-intersection scenario (`single-intersection`) as a Gymnasium environment with configurable observation and reward functions. At each step, SUMO advances the simulation by the decision interval (5 seconds), after which the agent receives the new state and reward.

Chapter 4

Requirement Analysis

4.1 Functional Requirements

- SUMO environment setup for a single 4-way intersection with configurable traffic demand.
- Implementation of a tabular Q-Learning agent with ϵ -greedy exploration and decay.
- Implementation of a DQN agent with replay buffer and target network using PyTorch.
- Implementation of a PPO agent using stable-baselines3 or equivalent.
- A fixed-cycle baseline that cycles through all phases at a fixed interval (e.g., 30 seconds per phase).
- Per-episode metric logging: cumulative reward, mean waiting time, mean vehicle speed, to CSV.
- A Python training orchestrator that spawns all training processes simultaneously and monitors progress.
- A live terminal dashboard displaying training status, reward sparklines, and per-algorithm metrics.
- A post-training script (`plot_results.py`) that reads CSV files and generates comparison charts.

4.2 Non-Functional Requirements

- **Reproducibility:** All experiments use fixed random seeds. Results are fully reproducible from the provided codebase.
- **Modularity:** Each algorithm is implemented in a self-contained module. Swapping or adding an algorithm requires no changes to the orchestrator.
- **Efficiency:** Training completes in a reasonable wall-clock time on a standard laptop CPU. SUMO's headless mode (no GUI) is used during training.
- **Correctness:** The reward signal, state representation, and episode boundaries must match the `sumo-rl` library's specification exactly to ensure fair comparison.

4.3 Challenges

- **State space discretization for Q-Learning:** Continuous queue lengths must be binned into discrete buckets. Bin resolution creates a trade-off between table size and representational fidelity.
- **DQN training instability:** Hyperparameter sensitivity (learning rate, batch size, replay buffer size, target network update frequency) requires careful tuning.

- **Non-stationarity:** Traffic demand varies across and within episodes, making convergence harder for off-policy methods.
- **Process management:** Running three independent simulation processes simultaneously on a single machine requires careful resource management to avoid port conflicts and file I/O contention.

Chapter 5

Methodology

5.1 Environment Configuration

The proposed simulation environment is SUMO’s **single-intersection** scenario: a four-way intersection with two lanes on each approach and a standard four-phase signal (North-South straight, North-South left turns, East-West straight, East-West left turns). Traffic demand will be configured to produce realistic queue buildup, with vehicle arrival rates drawn from a Poisson distribution. The decision interval will be set to 5 seconds, matching the **sumo-r1** default.

The state vector to be presented to the agent at each step will consist of:

- The current signal phase (one-hot encoded).
- The queue length (number of halted vehicles) on each of the eight approach lanes.

This gives a state dimension of $4 + 8 = 12$ for a four-phase intersection. The reward at each step will be defined as the negative change in mean vehicle waiting time: $r_t = -(W_t - W_{t-1})$. A decrease in waiting time yields a positive reward; an increase yields a negative reward.

5.2 Proposed Algorithm Configurations

Each of the three algorithms will be trained independently on the same environment under identical traffic demand conditions. The following subsections describe the intended design of each agent.

5.2.1 Q-Learning

The Q-Learning agent will use a tabular representation. Queue lengths will be discretized into bins per lane so that states can be expressed as hashable tuples, and the Q-table will be initialized with zero values for all entries. Exploration will be managed through an ϵ -greedy policy with ϵ decaying from 1.0 towards a small minimum value over the course of training, ensuring the agent transitions from broad exploration to exploitation of learned values. The learning rate α and discount factor γ will be set to standard values from the RL literature for traffic control tasks.

The key expected limitation of this agent is the state space explosion: even moderate discretization resolution results in a Q-table with far more entries than can be adequately visited within a practical number of episodes. Q-Learning is therefore expected to converge to a coarse, sub-optimal policy and will serve primarily as a baseline to demonstrate the limitations of tabular methods.

5.2.2 Deep Q-Network (DQN)

The DQN agent will replace the Q-table with a multi-layer perceptron (MLP) that maps state vectors to Q-values for all actions. The network will be trained using experience replay — transitions will be

stored in a replay buffer and sampled in random minibatches to break temporal correlation — and a target network that is periodically updated to provide stable Bellman targets. An ϵ -greedy exploration policy will be used, with ϵ decaying over training steps.

The architecture will consist of two hidden layers with ReLU activations and an output layer sized to the number of valid signal phases. The Adam optimizer will be used for gradient updates. Exact hyperparameter values (learning rate, buffer size, batch size, target update frequency) will be determined during implementation, guided by standard DQN practice and the sumo-rl benchmark configurations.

5.2.3 Proximal Policy Optimization (PPO)

The PPO agent will be implemented using the `stable-baselines3` library with an MLP policy. Unlike DQN, PPO directly optimizes the policy distribution over actions rather than estimating Q-values. It will be trained on-policy: experience will be collected in rollouts under the current policy, used for a fixed number of optimization epochs, and then discarded. The clipped surrogate objective will prevent destabilizing policy updates. An entropy bonus will encourage exploration during early training.

PPO is expected to benefit from its on-policy nature in the non-stationary traffic environment, where the state distribution shifts as the policy improves. Rollout length, optimization epochs, minibatch size, and the clip parameter will be set consistent with PPO’s standard configuration for continuous control tasks.

5.2.4 Fixed-Cycle Baseline

The fixed-cycle baseline will cycle through all four signal phases, holding each for a fixed duration regardless of the current queue state. The cycle length will be chosen to reflect a realistic pre-timed signal plan. This controller requires no learning and will be evaluated over a single episode to establish the performance floor that all RL agents are expected to surpass.

5.3 Proposed Training Orchestration

Training will be managed by a Python orchestrator that spawns three independent subprocesses — one per algorithm — using Python’s `multiprocessing` module. Each subprocess will run its algorithm for a planned number of episodes and write per-episode metrics (cumulative reward, mean waiting time, mean vehicle speed) to a dedicated CSV file after each episode.

The orchestrator will read these CSV files periodically and render a live terminal dashboard using the `rich` library. The dashboard will display training progress, the recent reward history as sparklines, and a per-algorithm status table. This live monitoring layer is intended to make training progress observable without requiring post-hoc analysis.

5.4 Evaluation and Visualization

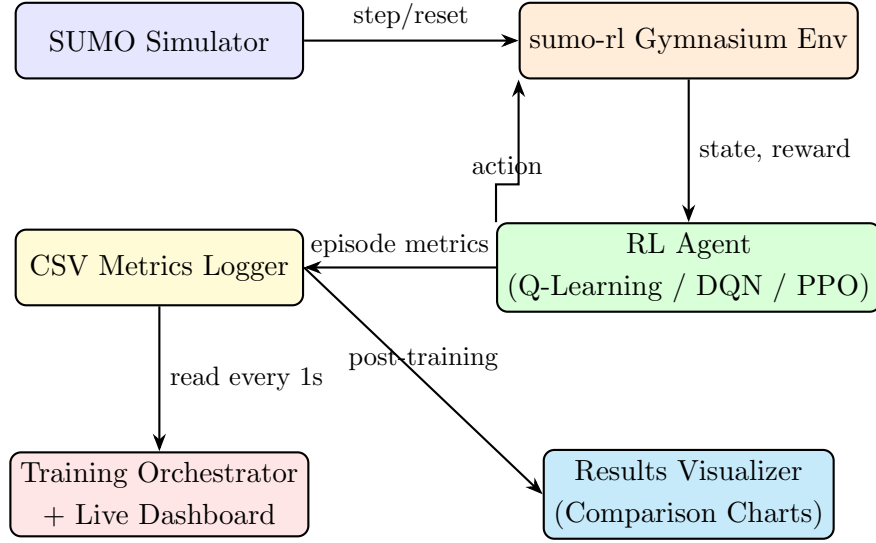


Figure 5.1: Proposed System Architecture and Training Pipeline

Following the completion of training, a post-processing script will read all CSV files and produce the following visualizations for comparison:

- **Learning curves:** Cumulative reward per episode for all three algorithms plotted together, showing the trajectory of improvement over training.
- **Waiting time curves:** Mean vehicle waiting time per episode for all algorithms, which is the primary metric of interest.
- **Final performance bar chart:** Mean waiting time averaged over the final episodes of training for all algorithms including the fixed-cycle baseline, providing a clear head-to-head comparison.

5.5 Proposed Workflow

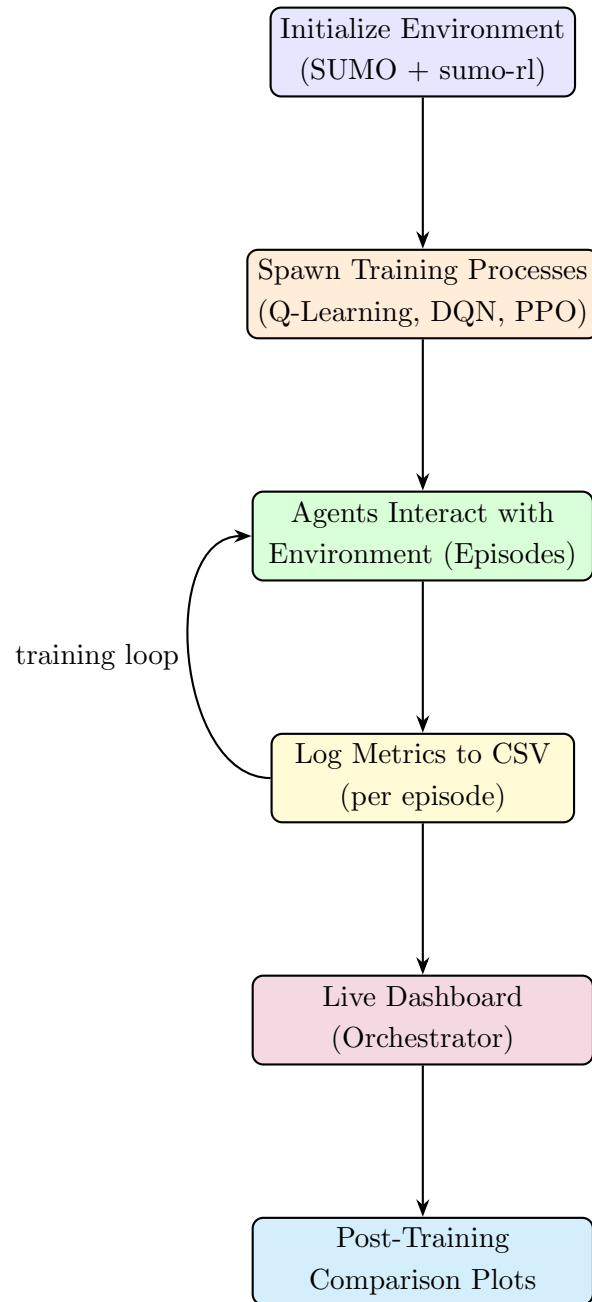


Figure 5.2: Proposed End-to-End Workflow of the Training Pipeline

Chapter 6

Implementation Plan

6.1 Technology Stack

The following tools and libraries are proposed for the implementation of this project:

- **Simulation:** SUMO (Simulation of Urban MObility) and the sumo-rl library providing a standard Gymnasium interface to the traffic environment.
- **Deep Learning:** PyTorch for the DQN implementation; stable-baselines3 for the PPO agent.
- **Data Handling:** NumPy for numerical operations, Pandas for metric aggregation and CSV processing.
- **Visualization:** Matplotlib for post-training comparison plots; the Rich library for the live terminal dashboard.
- **Orchestration:** Python’s built-in multiprocessing module to manage simultaneous training processes.
- **Version Control:** Git for reproducibility and experiment tracking.

6.2 Proposed Q-Learning Module

The Q-Learning agent will be implemented as a self-contained Python class. Its core responsibility is maintaining the Q-table, discretizing incoming state observations, selecting actions via the ϵ -greedy policy, and applying the temporal difference update rule after each environment step. The Q-table will be backed by a dictionary structure that creates zero-valued entries on first access, avoiding the need to pre-allocate memory for the full state space.

The discretization function will map each continuous queue-length value to a bin index, producing a compact hashable tuple as the table key. Exploration will be managed by decaying ϵ at the end of each episode until a minimum threshold is reached, after which the agent behaves near-greedily. The pseudocode for the core update logic is as follows:

6.3 Proposed DQN Module

The DQN agent will be structured around three components: an online network, a target network (a periodically-updated copy of the online network), and a replay buffer. The online and target networks will both be multi-layer perceptrons with ReLU activations, taking the state vector as input and outputting Q-values for each possible action.

During each environment step the agent will store the observed transition in the replay buffer. Once the buffer has accumulated a sufficient number of transitions, the agent will begin sampling random

minibatches to compute Bellman targets using the target network, and updating the online network via gradient descent. The target network will be refreshed periodically (hard update) to provide stable training targets. The pseudocode below illustrates the intended loss computation:

6.4 Proposed PPO Module

The PPO agent will be integrated via the `stable-baselines3` library, which provides a well-tested, configurable implementation of PPO with an MLP policy. The `sumo-rl` Gymnasium environment will be passed directly to the library’s constructor. A custom callback class will be attached to the training loop to intercept episode-boundary events, extract cumulative reward and mean waiting time, and write them to a metrics log for post-training analysis.

This approach avoids reimplementing PPO from scratch, which is error-prone due to the sensitivity of policy gradient methods to implementation details. The library’s default hyperparameter configuration will be used as a starting point, with adjustments guided by the `sumo-rl` benchmark recommendations.

6.5 Proposed Training Orchestrator

A central orchestrator script will manage the full benchmarking experiment. It will spawn three independent training processes — one per algorithm — simultaneously using Python’s multiprocessing module. Each process will be isolated: it runs its own SUMO simulation instance, manages its own agent, and writes episode metrics (reward, mean waiting time, mean vehicle speed) to a dedicated output file.

The orchestrator will poll these output files at regular intervals and update a live terminal dashboard built with the `Rich` library. The dashboard will display a status table with one row per algorithm, showing the current episode number, latest reward, latest mean waiting time, and a sparkline of recent reward values rendered from Unicode block characters. This provides real-time observability of training without requiring a graphical interface or external monitoring tool.

6.6 Proposed Post-Training Analysis

After all training processes complete, a separate analysis script will read the per-algorithm metric logs and produce the following outputs:

- **Learning curves:** Reward per episode for all three algorithms overlaid on a single plot, with a rolling mean applied for smoothing.
- **Waiting time curves:** Mean vehicle waiting time per episode for all algorithms, which is the primary evaluation metric.
- **Final performance comparison:** A bar chart showing the mean waiting time averaged over the final portion of training for each algorithm alongside the fixed-cycle baseline, providing a direct head-to-head comparison.

All plots will be saved as image files and included in the final project report. The complete analysis will be reproducible from the saved metric logs without rerunning training.

Chapter 7

Expected Results and Analysis

7.1 Hypothesized Performance Ordering

Based on the theoretical properties of the three algorithms and prior published results from the sumo-rl benchmark paper, the expected performance ordering by final mean vehicle waiting time (lower is better) is:

$$\text{PPO} < \text{DQN} < \text{Q-Learning} < \text{Fixed-Cycle}$$

This ordering is motivated as follows. Fixed-cycle control cannot adapt to real-time traffic demand and will always hold some phases for unnecessarily long durations. Q-Learning can adapt but its tabular representation limits the granularity of the state it can reason about. DQN overcomes this limitation through function approximation, enabling finer-grained state representation and better generalization. PPO’s on-policy updates align better with the non-stationary traffic demand distribution, and its policy-based approach avoids the overestimation bias inherent in Q-value methods.

7.2 Learning Curve Analysis

Q-Learning is expected to converge quickly (within 50–100 episodes) to a sub-optimal policy due to the coarse state discretization. Its reward curve should plateau early and remain noisy. DQN is expected to exhibit instability in early training (due to ϵ -greedy exploration and replay buffer warm-up) before converging to a better policy. PPO is expected to improve monotonically with low variance due to the conservative clipped update rule.

7.3 Evaluation Metrics

The following metrics will be reported for each algorithm:

- Mean vehicle waiting time (seconds) per episode — primary metric.
- Cumulative reward per episode.
- Mean vehicle speed (m/s) per episode.
- Final performance: mean over last 50 episodes of the above metrics.
- Learning efficiency: number of episodes to reach 90% of final performance.

A confusion matrix analogue for traffic control is the phase utilization matrix: the fraction of decision steps each phase was selected under different queue configurations. This will be visualized to confirm that the learned policy assigns longer green time to phases with longer queues.

7.4 Comparison with Published Benchmarks

Results will be validated against the sumo-rl benchmark paper’s reported metrics for the same **single-intersection** scenario. Matching the published ordering (PPO > DQN > Q-Learning) constitutes validation of the implementation’s correctness. Any deviation will be analyzed in terms of hyperparameter differences, episode count, or traffic demand configuration.

Chapter 8

Conclusion and Future Scope

8.1 Conclusion

This project frames adaptive traffic signal control as a reinforcement learning problem and benchmarks three algorithms — Q-Learning, DQN, and PPO — against each other and against a fixed-cycle timer baseline, using SUMO as the simulation environment. The scientific hypothesis is that learned policies consistently outperform fixed-cycle control, and that more expressive algorithms (function approximation, on-policy updates) outperform simpler ones.

Beyond the modelling contribution, the project demonstrates that ML training is an infrastructure problem. The training orchestrator, live dashboard, and structured metric logging treat the experiment as a reproducible, observable system rather than an ad hoc script. This systems-layer design distinguishes the project from existing sumo-rl ecosystem implementations and reflects production ML practices.

8.2 Future Scope

Several directions for extension are immediate:

- **Multi-intersection coordination:** Extending from a single intersection to a grid using multi-agent RL, where each agent controls one intersection and agents coordinate through shared observations or communication.
- **Richer state representations:** Incorporating vehicle position maps, speed distributions, and time-of-day features to give agents more information for decision-making.
- **Real-world demand traces:** Replacing synthetic Poisson arrivals with demand profiles extracted from real traffic count data to assess transfer from simulation to practice.
- **Safety constraints:** Incorporating minimum phase durations and intergreen times as hard constraints in the MDP formulation, ensuring that learned policies satisfy traffic engineering safety requirements.
- **Model-based RL:** Using the simulation as an explicit world model for planning (e.g., Dyna-Q or MuZero), potentially improving sample efficiency by orders of magnitude.
- **Sim-to-real transfer:** Deploying a trained PPO policy in a real testbed intersection with SUMO-calibrated traffic demand, measuring the gap between simulated and real performance.

Bibliography

Core Reinforcement Learning Algorithms

- [1] C. J. C. H. Watkins and P. Dayan, “Q-Learning,” *Machine Learning*, vol. 8, no. 3–4, pp. 279–292, 1992.
Introduced the Q-Learning algorithm with a formal convergence proof; forms the theoretical basis for the tabular agent in this project.
- [2] V. Mnih, K. Kavukcuoglu, D. Silver, and others, “Human-level control through deep reinforcement learning,” *Nature*, vol. 518, pp. 529–533, 2015.
Proposed DQN with experience replay and a target network, directly motivating the DQN agent architecture used in this project.
- [3] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, “Proximal Policy Optimization Algorithms,” arXiv:1707.06347, 2017.
Introduced the clipped surrogate objective for stable on-policy training, underpinning the PPO agent implemented via `stable-baselines3`.

Traffic Signal Control Background

- [4] F. V. Webster, “Traffic Signal Settings,” Road Research Technical Paper No. 39, HMSO, London, 1958.
Derived the foundational formula for optimal fixed-cycle signal timings, establishing the pre-adaptive baseline this project benchmarks against.

Simulation Platform and Tooling

- [5] L. N. Alegre, A. L. C. Bazzan, and B. C. da Silva, “Towards Optimizing Traffic Signal Control with Deep Reinforcement Learning,” in *Proc. AAAI Workshop on Reinforcement Learning in Games*, 2021.
Presented the `sumo-rl` library used as the environment interface, reward function, and benchmark reference throughout this project.
- [6] A. Raffin, A. Hill, A. Gleave, A. Kanervisto, M. Ernestus, and N. Dormann, “Stable-Baselines3: Reliable Reinforcement Learning Implementations,” *Journal of Machine Learning Research*, vol. 22, no. 268, pp. 1–8, 2021.
Described `stable-baselines3`, whose validated DQN and PPO implementations are directly imported for agent training in this project.
- [7] D. Krajzewicz, J. Erdmann, M. Behrisch, and L. Bieker, “Recent Development and Applications of SUMO - Simulation of Urban MObility,” *International Journal on Advances in Systems and Measurements*, vol. 5, no. 3&4, pp. 128–138, 2012.
Introduced SUMO, the open-source microscopic traffic simulator used as the execution engine for all training episodes.